

Probability and Combinatorics in Bingo

Michael Stafford

April 2026

1 Introduction

The purpose of this write-up is to answer the following question: what is the expected number of turns before a bingo occurs?

To answer this question, we start by considering the number of markers on the board and the probability of getting bingo from each number of markers. From there, we use an algorithm to determine the number of bingo layouts and the negative hypergeometric distribution to calculate our final value.

2 Definitions and Setup

Throughout this paper, let X be a random variable representing the number of turns that transpire until a bingo occurs and let Z represent the number of markers on the board when a bingo occurs. Therefore, our research question is solving for $\mathbb{E}[X]$. As mentioned in the introduction, we start by considering the probability distribution on Z and build from there.

The following assumptions are used for the remainder of the paper:

1. There is no "free space" on the board. In other words, the board starts completely blank.
2. There are 75 numbers that can be called and 25 numbers on a board.
3. Each number has an equal probability of being called during every turn. Numbers cannot be called more than once.
4. **Critical assumption:** once a bingo is achieved, the game ends. No markers are added to the board after a bingo.

Section 6 often refers to "layouts". By "layout", we mean an arrangement of tiles on a bingo board. A more formal mathematical definition is provided in Section 6.

3 Simulator

First, I constructed a program that simulates bingo games. By simulating a large number of Bingo games, we converge on $\mathbb{E}[X]$ and $\mathbb{E}[Z]$ using mean estimators $\bar{x}$ and $\bar{z}$ respectively. By running the simulation several times, I obtained the following results.

Table 1: Simulated Values

	$n = 10^2$	$n = 10^3$	$n = 10^4$	$n = 10^5$
$\bar{x}$	44.75	43.522	43.7279	43.55898
$\bar{z}$	15.24	14.916	14.9337	14.90944

These results provide a solid approximation for $\mathbb{E}[X]$ and $\mathbb{E}[Z]$ and a standard against which to check a more analytical, principles-based solution.

4 Formula for $\mathbb{E}[X]$

We proceed by determining the probability distribution on X , which will in turn lead us to $\mathbb{E}[X]$.

The support of X can be deduced logically. $\text{supp}(X) \subset \mathbb{N}$, since X is representing a number of turns. The quickest possible bingo game results in $X = 5$. The longest possible game results in $X = 71$: this corresponds to calling all 50 numbers not on the board and requiring 21 tiles on the board (the maximum possible) in order to get a bingo. Therefore, the support of X is $[5, 71]$ and the support of Z is $[5, 21]$.

For any $x \in [5, 71]$, $P(X = x)$ can be calculated using the law of total probability in terms of Z . When each bingo game ends, there are a certain number of turns that have transpired, x , and a certain number of markers on the board, z . We can express $P(X = x)$ as:

$$P(X = x) = P(X = x|Z = 5) \cdot P(Z = 5) + \dots + P(X = x|Z = 21) \cdot P(Z = 21)$$

$$P(X = x) = \sum_{z=5}^{\min(x, 21)} P(X = x|Z = z) \cdot P(Z = z)$$

Note that when $x < 21$, summation terms for which $z > x$ will automatically be 0 (you can't have more markers than turns). Therefore, we can constrain the limit on the summation to be the minimum of x and 21.

Once $P(X = x)$ can be calculated, finding $\mathbb{E}[X]$ is straightforward:

$$\mathbb{E}[X] = \sum_{x=5}^{71} P(X = x) \cdot x$$

In the next section, we calculate $P(X = x|Z = z)$ using the negative hypergeometric distribution. Then, we use an algorithm to determine $P(Z = z)$.

5 $P(X = x|Z = z)$ and the Negative Hypergeometric Distribution

A bingo game is a classic example of sampling without replacement: once a number is called, it cannot be called again for the rest of the game. The negative hypergeometric distribution is used to model the number of "failures" that occur before a certain number of "successes" in a series of draws without replacement. In this bingo scenario, a "failure" means a number called that is not on the board, while a "success" is a number called that is on the board. Intuitively, $P(X = x|Z = z)$ means "what is the probability of the bingo game taking x turns, given that it takes z tiles on the board to get a bingo?"

For a random variable H that is distributed according to the negative hypergeometric distribution, the probability mass function is given by

$$P(H = k) = \frac{\binom{k+z-1}{k} \binom{N-z-k}{K-k}}{\binom{N}{K}}$$

Where z represents the predetermined number of successes, k represents the number of failures, K represents the total number of failure states, and N corresponds to the total number of states.

In our case, the "states" are bingo numbers. The z in the PMF matches the z in our example. The number of failures k corresponds to the balls that are called that are not on the board, which is $x - z$, the number of failure states K is the number of bingo balls not on our board, which are 50, and N is the total number of bingo balls, which are 75. With this setup, the expression $P(X = x|Z = z)$ is given by

$$P(X = x|Z = z) = \frac{\binom{x-1}{x-z} \binom{75-x}{50-(x-z)}}{\binom{75}{50}}$$

6 $P(Z = z)$ and Layout Classification

Recall that $P(Z = z)$ means "the probability that bingo is achieved with z markers on the board". The primary complication with the bingo problem is that not all layouts are possible and not all layouts occur with equal probability. Therefore, we divide layouts into categories, depending on their possibility and permutations. Using these categories and their associated permutations, we can construct $P(Z = z)$ by counting outcomes. First, we need to establish a few definitions.

Definition : A **layout** ℓ is a 5x5 matrix, where each entry $[m_{ij}] \in \{0, 1\}$. An entry of 1 denotes a filled square. A layout with z number of tiles is denoted ℓ_z

Definition : Let B denote the set of all layouts with a bingo.

Definition : Let $\ell \in B$. A **keystone** is an entry $[m_{ij}]$ such that if $[m_{ij}]$ were set to 0, $\ell \notin B$.

In essence, a keystone is a marker in a bingo layout such that if the marker were to be removed, the layout would no longer have a bingo. By necessity of our critical assumption, a keystone must be the last marker placed. Otherwise, a bingo would have already occurred and the game was already over!

With these definitions in mind, we can place every layout in one of four categories:

1. Invalid layout (bingo layout with no keystone)
2. Bingo layout with a unique keystone
3. Bingo layout without a unique keystone
4. Not a bingo layout

6.1 Invalid Layout

An invalid layout is a layout that, under our critical assumption, cannot occur. Consider the following example:

$$\ell = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

This layout is impossible because in order to get to this point, a bingo had to have been achieved previously, so the game was already over. The formal definition of an invalid layout is as follows:

Definition : A bingo layout $\ell \in B$ is **invalid** if it has no keystones. In other words, there is no entry $[m_{ij}]$ such that if $[m_{ij}]$ were set to 0, $\ell \notin B$.

Let I denote the set of all invalid layouts. Invalid layouts never occur during a game of bingo. Therefore, the algorithm skips over these layouts in its calculations.

6.2 Bingo Layout with a Unique Keystone

Some bingo layouts have exactly one keystone. The set of all such layouts will be denoted U . Here is an example of such a layout $\ell_9 \in U$:

$$\ell = \begin{bmatrix} \mathbf{1} & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

In this example, the keystone marker is the top left entry, indicated in bold. For a layout $\ell_z \in U$, note that the unique keystone *must be the last marker added*, otherwise the game would already have been over. Therefore, the number of ways in which a layout $\ell_z \in U$ could occur is $(z-1)!$ The keystone marker is constrained to be last, but all other markers could be placed in any order. The number of bingo layouts with a unique keystone with z tiles is denoted by a_z .

6.3 Bingo Layout without a Unique Keystone

Some bingo layouts have multiple keystones. Any layout with more than one keystone has 5 total keystones. These keystones are the markers that constitute the bingo in the arrangement. The set of all bingo layouts with multiple keystones is denoted M . Here is an example of a layout $\ell_9 \in M$:

$$\ell = \begin{bmatrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

For a layout $\ell_z \in M$ note that *one of the keystones* must be the last marker added. It doesn't matter which of the 5 keystones are last. Therefore, the number of ways in which a layout $\ell_z \in M$ could occur is $5(z-1)!$ One of the 5 keystones must be last, but the other markers are unconstrained, giving us 5 ways of arranging $(z-1)!$ markers. The number of bingo layouts without a unique keystone with z tiles is denoted by b_z .

6.4 Not a bingo layout

Let N denote the set of layouts that do not result in a bingo. Here is an example of a layout $\ell_9 \in N$:

$$\ell = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Each marker in $\ell_z \in N$ can go in any order. The markers are not constrained by any invalid combinations. Therefore, the number of ways in which a layout

$\ell_z \in N$ could occur is $z!$. The number of non-bingo layouts with z tiles is denoted by c_z .

Table 2: Summary of Layout Categories

Category	Term	Permutations
Invalid	N/A	0
Bingo: Unique Keystone	a_z	$(z - 1)!$
Bingo: Non-unique Keystone	b_z	$5(z - 1)!$
Non-bingo	c_z	$z!$

6.5 Algorithm

The purpose of the algorithm is to found the values of (a_z, b_z, c_z) for each $z \in [5, 21]$. The algorithm iterates through layouts and categorizes each into one of the four categories mentioned above. The output of the algorithm is a dictionary that maps the number of tiles z to the ordered triple (a_z, b_z, c_z) . As mentioned earlier, invalid layouts are ignored by the algorithm.

I implemented the algorithm in two ways: one using a brute-force method (iterating through all 2^{25} layouts) and one using a recursive method to iterate through the layouts. Both algorithms have the same result. The brute-force method is actually slightly faster, but the recursive method would generalize better for larger boards. Both take a little over 30 minutes.

Regardless of how the algorithm iterates through layouts, the process is the same for each layout. Given a layout ℓ :

1. Calculate z by counting all the markers in the layout.
2. If $\ell \notin B$, increment c_z and return (move on to the next layout).
3. Otherwise, define a variable k representing the number of keystones, initialized at 0.
4. Iterate through each filled entry $[m_{ij}] = 1$ in ℓ :
 - (a) Define a new layout ℓ^* that equals ℓ in every entry except $[m_{ij}] = 0$.
 - (b) If $\ell^* \notin B$, increment k
5. if $k = 0$, this is an invalid layout.
 else if $k = 1$, increment a_z .
 else if $k > 1$, increment b_z .

By running the algorithm, I obtained the following results:

Table 3: Algorithm Results

z	a_z	b_z	c_z
5	0	12	53118
6	0	250	176860
7	0	2280	478420
8	0	13680	1067895
9	46	58048	1984881
10	736	184536	3083468
11	5520	453480	3998100
12	25616	874664	4297872
13	81544	1328328	3780704
14	186096	1575736	2664548
15	308388	1430612	1456572
16	367120	958328	587935
17	303041	445644	162362
18	161872	129888	27088
19	49332	19788	2176
20	6776	1064	48
21	240	0	0

6.6 Compiling Algorithm Results into $P(Z = z)$

(a_z, b_z, c_z) and their associated permutations help us determine the probability of getting bingo *given that there are z tiles on the board*, but we also need to calculate the probability that z tiles made it onto the board in the first place. Therefore, the $P(Z = z)$ calculation also requires factoring in our critical assumption: in order to achieve bingo with exactly z tiles, all previous tiles arranges from 1 to $z - 1$ must have failed to achieve bingo.

$$P(Z = z) = P(Z \neq 1, 2, \dots, z - 1) \cdot P(Z = z | Z \neq 1, 2, \dots, z - 1)$$

Note however, that for $z = 5$, the probability calculation is more simple. The probability of bingo for $z \in [0, 1, 2, 3, 4]$ is 0, so we can calculate $P(Z = 5)$ by simply counting the number of possible bingos with 5 tiles (12) and dividing by the number of possible ways to populate a 25-square board with 5 tiles $\binom{25}{5}$. Therefore, $P(Z = 5) = \frac{5}{\binom{25}{5}}$.

By applying DeMorgan's law and dropping the terms less than 5, we get:

$$P(Z = 5) = \frac{5}{\binom{25}{5}}$$

$$P(Z = z) = [1 - \sum_{j=5}^{z-1} P(Z = j)] \cdot P(Z = z | Z \neq 1, 2, \dots, z - 1), \forall z > 5$$

$P(Z = z|Z \neq 1, 2, \dots, z-1)$ is calculated using the results of the algorithm. a_z and b_z are bingo layouts with permutations of $(k-1)!$ and $5(k-1)!$ respectively. These values will go in the numerator. The denominator will also contain these values along with c_z and its $k!$ permutations. Thus, our equation is

$$P(Z = 5) = \frac{5}{\binom{25}{5}}$$

$$P(Z = z) = [1 - \sum_{j=5}^{z-1} P(Z = j)] \cdot \frac{a_z(z-1)! + 5b_z(z-1)!}{a_z(z-1)! + 5b_z(z-1)! + c_z z!}, \forall z > 5$$

or to simplify things slightly,

$$P(Z = 5) = \frac{5}{\binom{25}{5}}$$

$$P(Z = z) = (z-1)! \cdot [1 - \sum_{j=5}^{z-1} P(Z = j)] \cdot \frac{a_z + 5b_z}{a_z(z-1)! + 5b_z(z-1)! + c_z z!}, \forall z > 5$$

By substituting the actual values into this expression, we get

$$\mathbb{E}[X] \approx 43.54558139785762$$

$$\mathbb{E}[Z] \approx 14.897172583477603$$